

Documentação de Projeto

para o sistema

AutoFix

Versão 1.0

Projeto de sistema elaborado pelo(s) aluno(s) Sofia Vasconcelos Moreira e Silva
como parte da disciplina **Projeto de Software**.

04/06/2026

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso e Histórias de Usuários	1
2.3 Diagrama de Sequência do Sistema e Contrato de Operações	1
3. Modelos de Projeto	1
3.1 Arquitetura	1
3.2 Diagrama de Componentes e Implantação.	2
3.3 Diagrama de Classes	2
3.4 Diagramas de Sequência	2
3.5 Diagramas de Comunicação	2
3.6 Diagramas de Estados	2
4. Modelos de Dados	2

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Sofia V. M. e Silva	25/05/2026	Criação do documento, especificação de requisitos e modelo de casos de uso.	V 0.1
Sofia V. M. e Silva	30/05/2026	Inclusão dos diagramas de arquitetura (C4), classes, sequência e esquema do banco de dados.	V 0.2
Sofia V. M. e Silva	04/06/2026	Revisão geral, refinamento das estratégias de ORM e finalização para entrega	V 1.0

1. Introdução

Este documento agrega a elaboração, revisão de modelos de domínio e modelos de projeto para o sistema **AutoFix**. O sistema visa digitalizar e otimizar o fluxo de trabalho de uma oficina mecânica, cobrindo desde a entrada do veículo, abertura de ordens de serviço (OS), diagnóstico dos mecânicos, até a finalização e faturamento.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

- **Cliente:** Pessoa física ou jurídica que solicita os serviços de manutenção ou revisão de veículos.
- **Recepcionista:** Responsável pelo primeiro contato, cadastro de clientes/veículos e abertura inicial da Ordem de Serviço (OS).
- **Mecânico:** Profissional técnico que realiza o diagnóstico do veículo, inclui as peças/serviços necessários na OS e executa o trabalho.
- **Gerente:** Responsável por aprovar orçamentos complexos, gerenciar relatórios financeiros e estoque de peças.

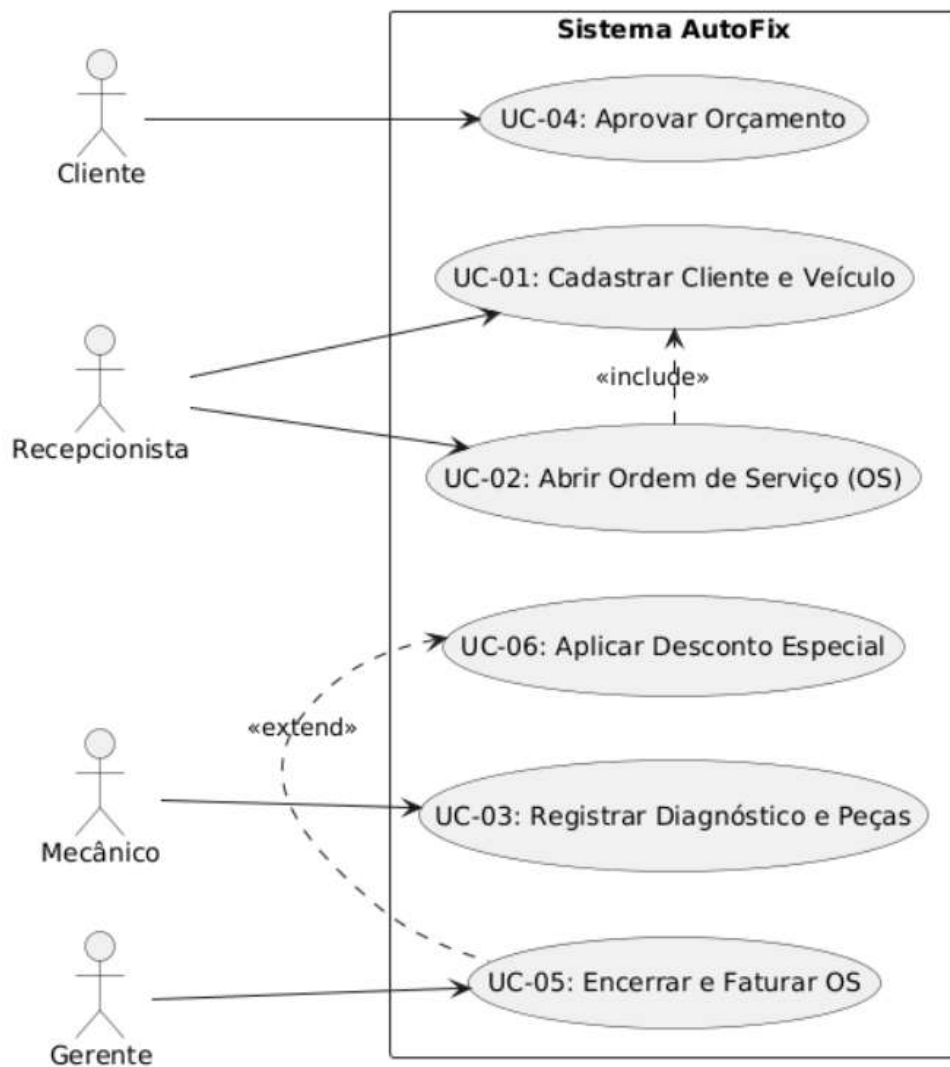
2.2 Modelo de Casos de Uso

O diagrama de casos de uso aborda as principais funcionalidades do AutoFix, estruturando de forma clara as interações entre os atores (Clientes, Recepcionistas, Mecânicos e Gerentes) e as operações essenciais do sistema. Ele delimita o escopo do projeto, mapeando desde o fluxo inicial de cadastro e abertura de Ordens de Serviço até as etapas críticas de diagnóstico técnico, aprovação pelo cliente e o encerramento financeiro, garantindo uma visão holística e funcional do ecossistema da oficina.

Histórias de Usuário (User Stories)

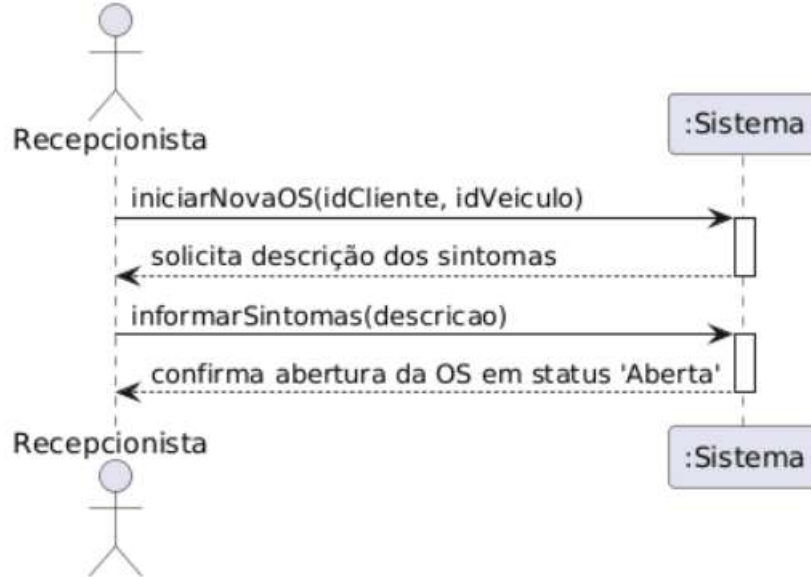
- **US-01 (Cadastrar Cliente e Veículo):** Como **Recepcionista**, eu quero cadastrar os dados pessoais do cliente e os dados do seu veículo no sistema, para que as informações fiquem salvas para atendimentos futuros.

- **US-02 (Abrir Ordem de Serviço):** Como **Recepcionista**, eu quero abrir uma nova Ordem de Serviço associando o cliente ao veículo e descrevendo os sintomas relatados, para que o mecânico saiba o que precisa ser avaliado.
- **US-03 (Registrar Diagnóstico e Peças):** Como **Mecânico**, eu quero registrar o diagnóstico técnico e adicionar as peças e serviços necessários na Ordem de Serviço, para que o sistema calcule o valor parcial do orçamento.
- **US-04 (Aprovar Orçamento):** Como **Cliente**, eu quero visualizar o orçamento detalhado da manutenção do meu veículo para aprovar ou rejeitar a execução dos serviços com transparência.
- **US-05 (Encerrar e Faturar OS):** Como **Gerente**, eu quero encerrar a Ordem de Serviço após a execução, registrar o método de pagamento e emitir o recibo, para registrar a entrada financeira no caixa e dar baixa nas peças do estoque.

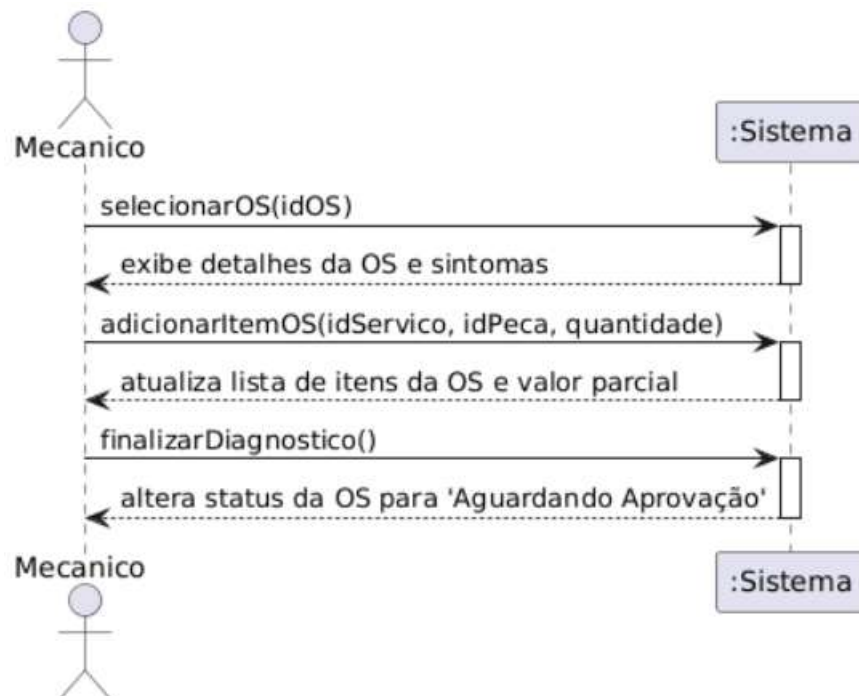


2.3 Diagrama de Sequência do Sistema

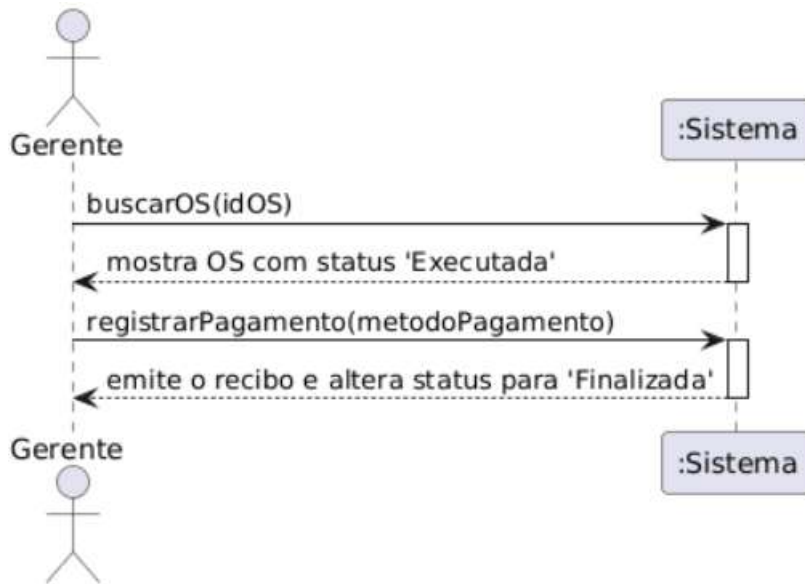
DSS - UC-02: Abrir Ordem de Serviço (OS):



DSS - UC-03: Registrar Diagnóstico e Peças:



DSS - UC-05: Encerrar e Faturar OS:



Operação de Abertura de Ordem de Serviço

Contrato	CO-01: iniciarNovaOS
Operação	iniciarNovaOS(idCliente, idVeiculo)
Referências cruzadas	UC-02: Abrir Ordem de Serviço (OS)
Pré-condições	O Cliente e o Veículo já devem estar previamente cadastrados no sistema.
Pós-condições	- Uma nova instância da classe OrdemServico foi criada. - A OrdemServico foi associada ao Cliente e ao Veículo. - O status inicial da OS foi definido como "Aberta".

Operação de Adição de Itens pelo Mecânico

Contrato	CO-02: adicionarItemOS
Operação	adicionarItemOS(idServico, idPeca, quantidade)
Referências cruzadas	UC-03: Registrar Diagnóstico e Peças
Pré-condições	Uma Ordem de Serviço válida deve estar selecionada e ativa em estado de "Diagnóstico".
Pós-condições	- Uma nova instância de ItemOS foi criada. - O ItemOS foi associado à OrdemServico correspondente.

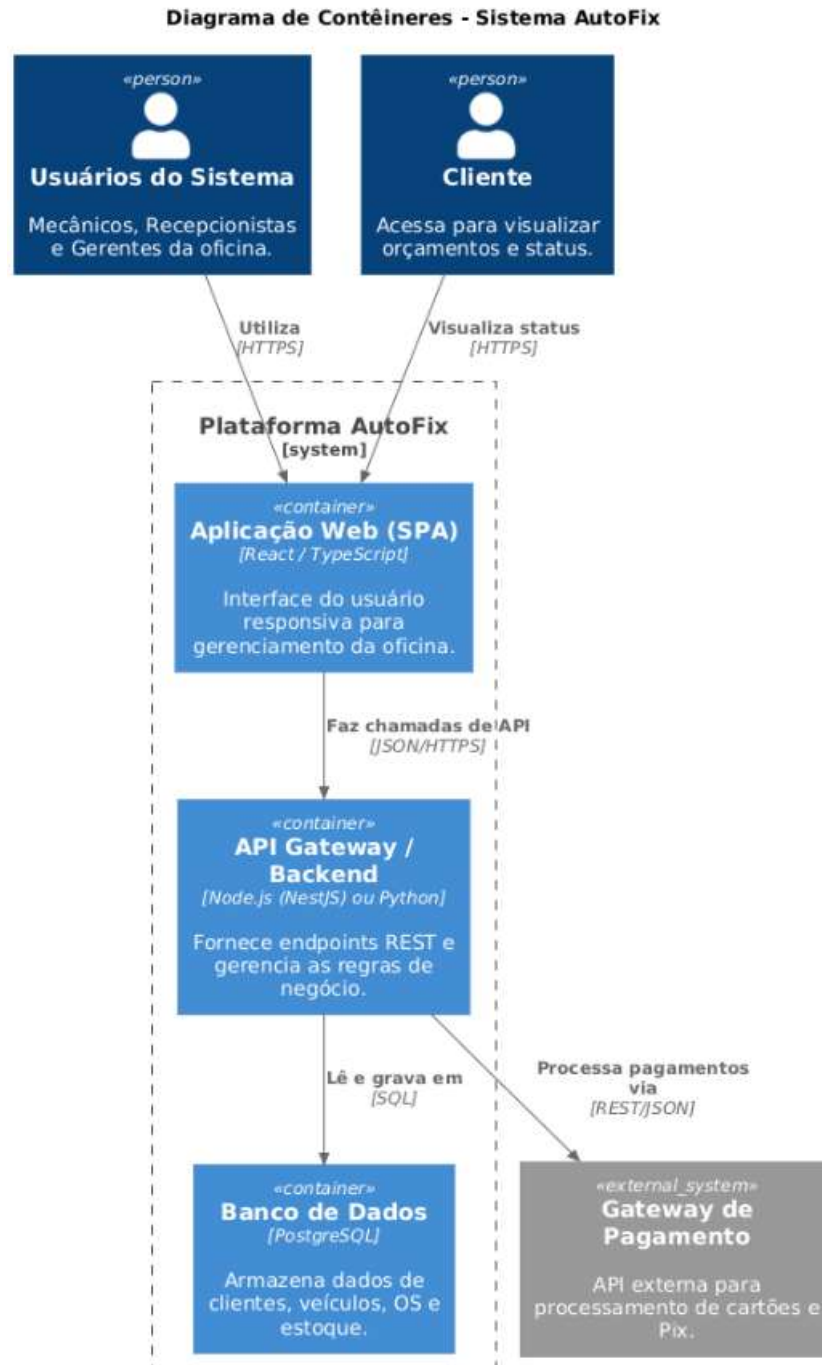
	- A quantidade de peças informada foi reservada temporariamente no estoque. - O valor total parcial da OS foi recalculado e atualizado.
--	---

Operação de Encerramento Financeiro

Contrato	CO-03: registrarPagamento
Operação	registrarPagamento(metodoPagamento)
Referências cruzadas	UC-05: Encerrar e Faturar OS
Pré-condições	A Ordem de Serviço deve estar com o status "Executada" (serviço concluído pelo mecânico).
Pós-condições	- O status da OrdemServico foi alterado para "Finalizada". - Um registro de movimentação financeira (receita) foi gerado no caixa. - As peças utilizadas foram baixadas definitivamente do estoque do sistema.

3. Modelos de Projeto

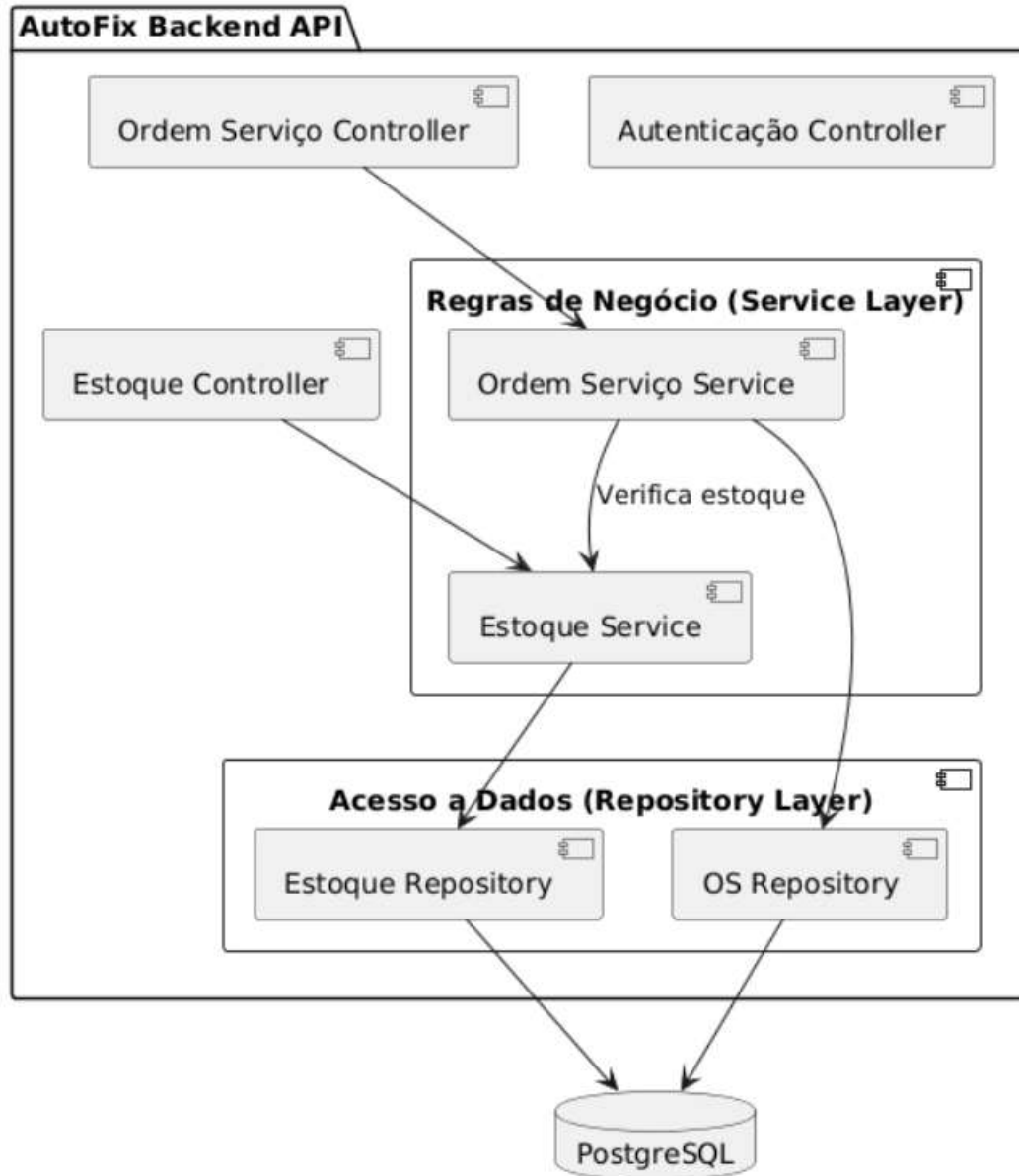
3.1 Arquitetura



3.2 Diagrama de Componentes e Implantação.

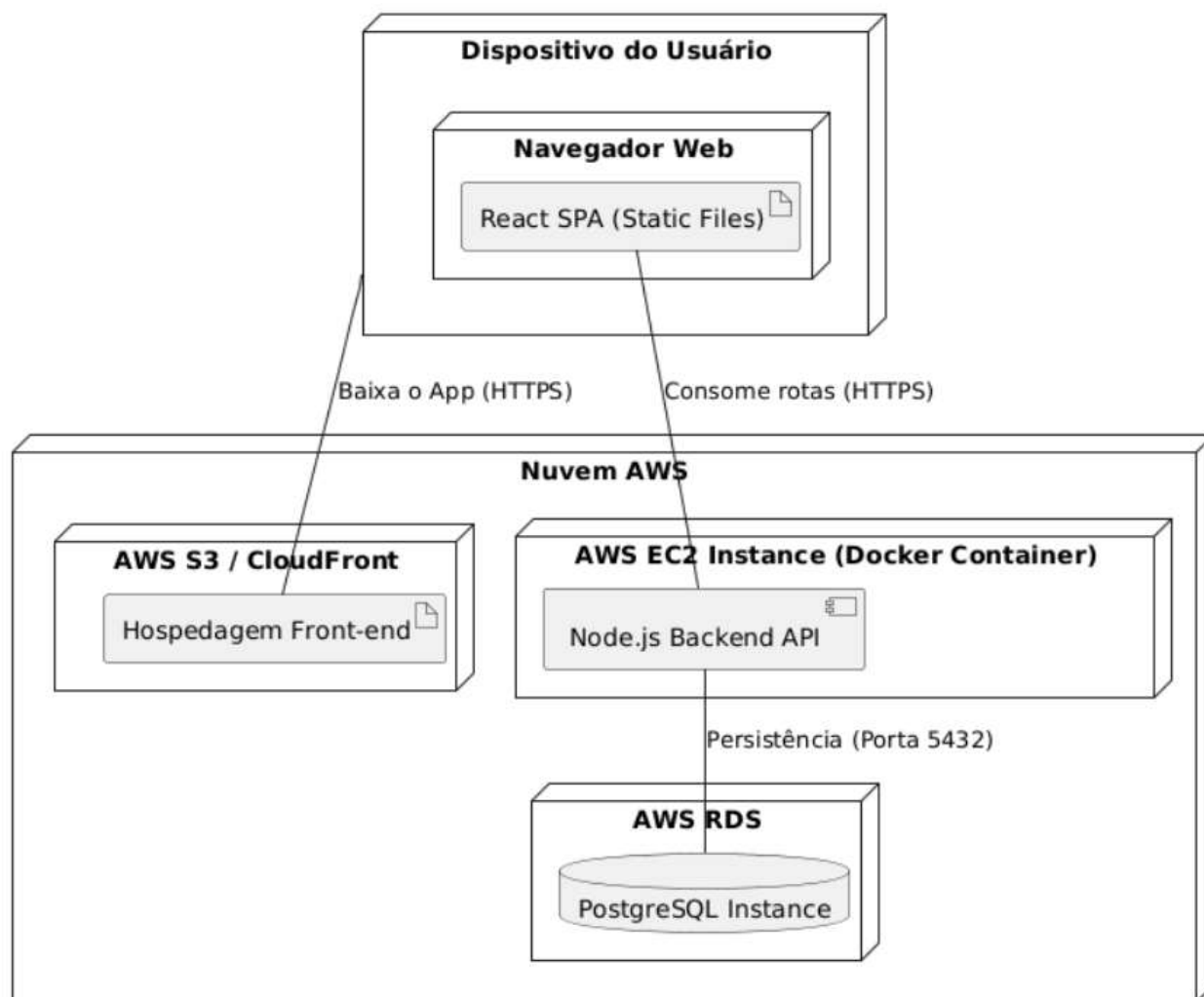
A) Diagrama de Componentes

Este diagrama detalha como o nosso contêiner de **Backend (API)** é estruturado internamente (as camadas de software).



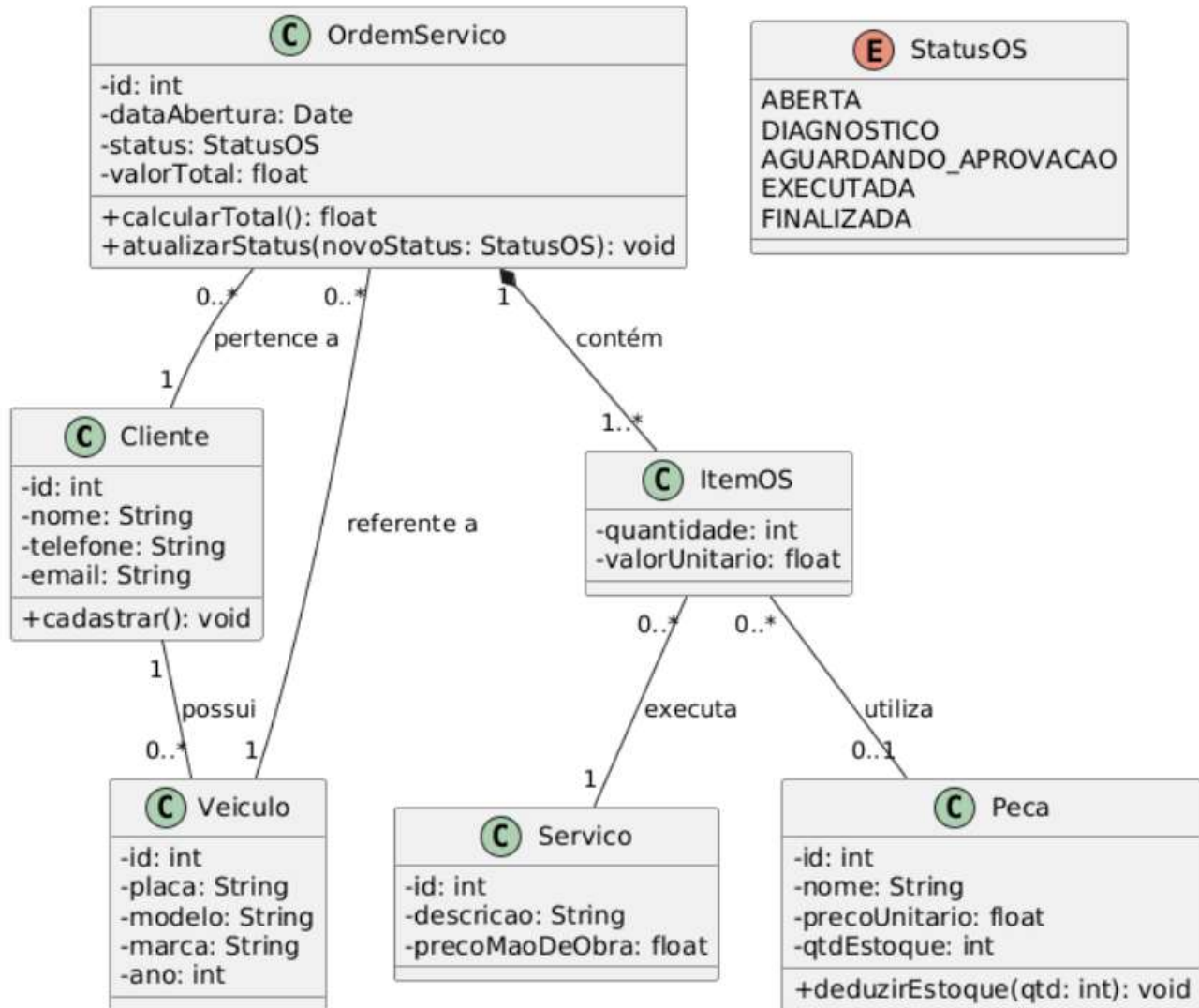
B) Diagrama de Implantação (Deployment)

Mostra onde o sistema roda fisicamente (infraestrutura na nuvem, como AWS).



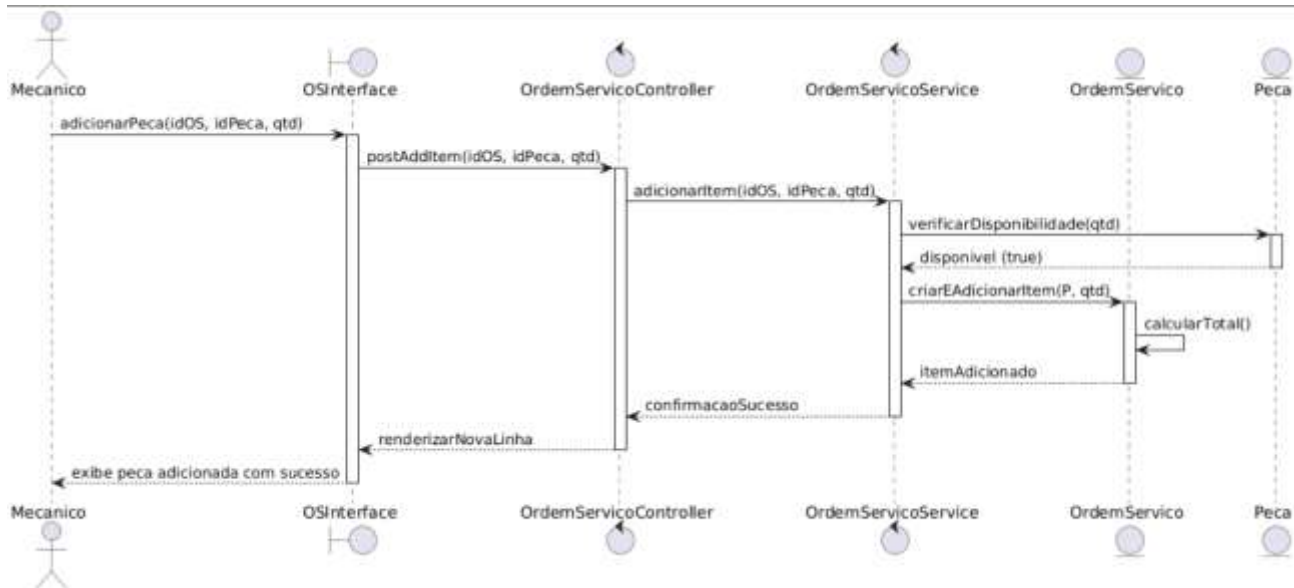
3.3 Diagrama de Classes

Este é o coração estrutural do projeto. Ele mapeia as entidades que vão virar tabelas e objetos no código.



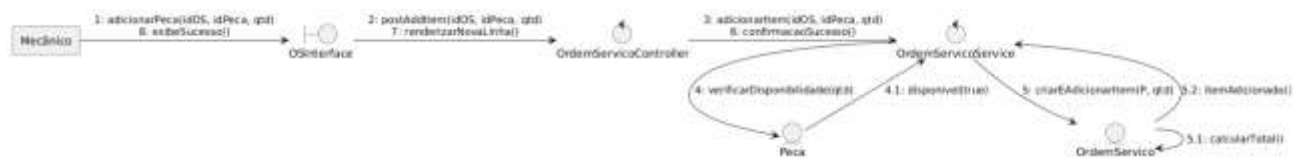
3.4 Diagramas de Sequência

O diagrama de sequência de projeto detalha a realização da operação crítica Registrar Diagnóstico, mapeando a troca de mensagens síncronas entre os objetos e componentes internos da API do AutoFix. Quando o ator Mecânico interage com a interface, o OrdemServicoController intercepta a requisição HTTP e delega as regras de negócio para a classe OrdemServicoService. Para garantir a integridade do fluxo, a camada de serviço invoca o EstoqueService para verificar a disponibilidade de componentes antes de associar novos itens à entidade OrdemServico. Por fim, os estados atualizados são persistidos de forma encapsulada através das classes de repositório (OSRepository e EstoqueRepository) no banco de dados PostgreSQL.



3.5 Diagramas de Comunicação

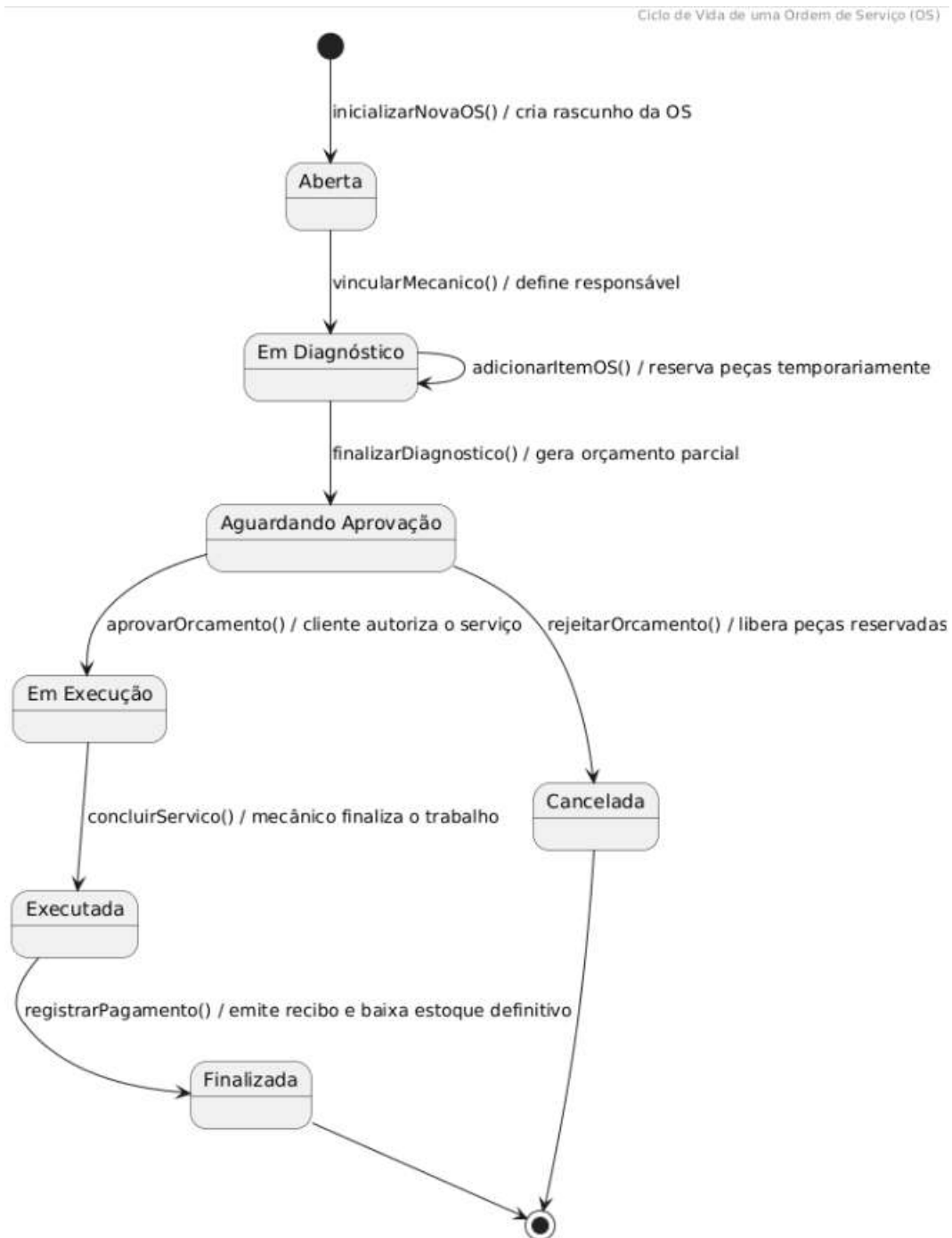
Complementando a visão dinâmica do sistema, o diagrama de comunicação modela o mesmo cenário de registro de diagnóstico técnico, porém concentrando-se na organização espacial e nos vínculos estruturais estabelecidos entre as instâncias em tempo de execução. Através de uma numeração sequencial explícita das mensagens (ex: 1, 1.1, 2), o diagrama deixa clara a topologia da arquitetura em camadas. Ele demonstra visualmente o acoplamento fraco e a alta coesão do padrão Boundary-Control-Entity, onde os objetos de controle coordenam o fluxo de dados sem que a camada de apresentação tenha conhecimento direto sobre as entidades de persistência ou os mecanismos de banco de dados.



3.6 Diagramas de Estados

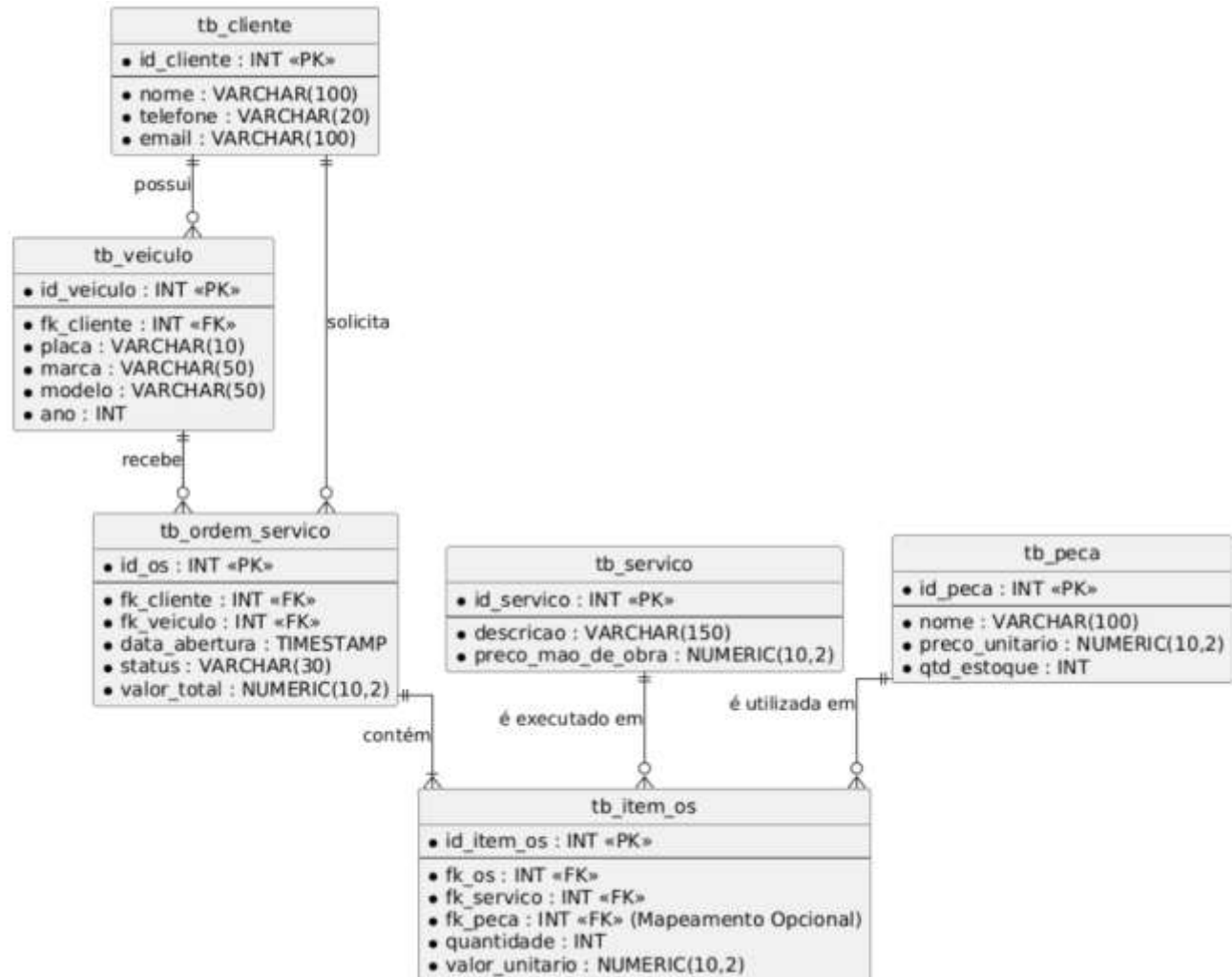
O ciclo de vida de uma Ordem de Serviço (OS) dentro da plataforma AutoFix é governado por uma máquina de estados finitos bem delimitada, essencial para garantir a consistência das regras de negócio e impedir faturamentos indevidos. Conforme ilustrado no diagrama a seguir, o ciclo inicia-se obrigatoriamente no estado Aberta. A partir da alocação de um técnico, transita para Em Diagnóstico, fase na qual itens podem ser agregados consecutivamente, disparando reservas temporárias de estoque. Uma vez concluída a análise pelo mecânico, a OS assume o estado Aguardando Aprovação. Caso o cliente rejeite as condições, o fluxo avança para Cancelada, liberando os insumos reservados; se aprovado, transita para Em Execução. Após o término físico dos reparos, passa para

Executada, aguardando a chamada de encerramento financeiro para consolidar-se permanentemente como Finalizada.



4. Modelos de Dados

4.1 Esquema de Banco de Dados (DER)



4.2 Estratégias de Mapeamento Objeto-Relacional (ORM)

Como a aplicação é estruturada utilizando o paradigma de Orientação a Objetos no back-end (ex: TypeScript/NestJS ou C#/.NET) e os dados são persistidos em um Banco de Dados Relacional (PostgreSQL), adota-se o padrão **ORM (Object-Relational Mapping)** para fazer a ponte entre esses dois mundos.

As estratégias de mapeamento configuradas para o AutoFix são:

1. Mapeamento de Tabelas e Classes:

- Cada entidade do diagrama de classes (ex: Cliente) é mapeada diretamente para uma tabela correspondente no banco de dados (`tb_cliente`) utilizando

anotações/decorators (como @Entity() do TypeORM ou [Table] do Entity Framework).

2. Mapeamento de Chaves e Identificadores:

- Todas as chaves primárias (<<PK>>) utilizam a estratégia de auto-incremento gerenciada pelo banco de dados (IDENTITY ou SERIAL), mapeadas no código como chaves geradas automaticamente.

3. Mapeamento de Associações (Relacionamentos):

- **Um-para-Muitos (1..*)**: O relacionamento entre Cliente e Veiculo é resolvido no banco através da chave estrangeira fk_cliente na tabela tb_veiculo. No código, isso é mapeado usando as propriedades @ManyToOne e @OneToMany.
- **Muitos-para-Muitos (..)**: A relação entre a Ordem de Serviço e os Serviços/Peças gerou a tabela associativa tb_item_os. No código, ela é tratada como uma entidade rica (pois possui atributos próprios como quantidade e valor_unitario), mapeada com relacionamentos intermediários de um-para-muitos.

4. Estratégia de Tipos de Dados:

- Valores monetários (valor_total, preco_unitario) utilizam o tipo NUMERIC(10,2) no banco de dados para evitar problemas de arredondamento de ponto flutuante, sendo convertidos para o tipo number ou decimal na aplicação.
- O ciclo de vida da OS (Status) é armazenado como VARCHAR no banco de dados para facilitar auditorias visuais diretas nas tabelas, mas é mapeado como um enum fortemente tipado no código da aplicação.